

Week 06: React State

2110507 Software Development Practice II

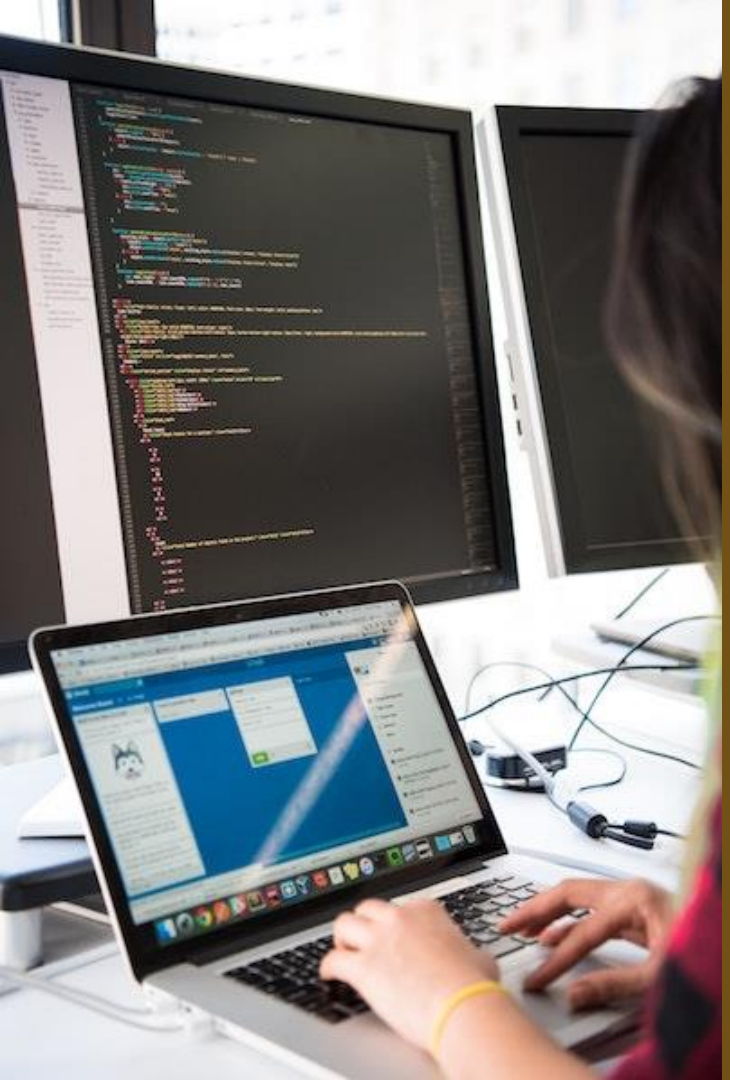
Machigar Ongtang, Ph.D.

State



- State เป็น object ใน React ที่จัดเก็บสถานะปัจจุบัน data
- State เป็น memory ที่จดจำในระดับ component (component-specific memory)
  - ใช้ในการบันทึกค่า property สำหรับ component
  - ใช้จดจำ data ต่าง ๆ เช่น user input, สถานะชั่วคราวของ shopping cart เป็นต้น
  - เมื่อเกิดการเปลี่ยนแปลงขึ้นกับ state object จะมีการ re-render component นั้น ๆ
  - state ของ component สามารถเข้าถึงจากที่ใดใน component นั้น ๆ ก็ได้

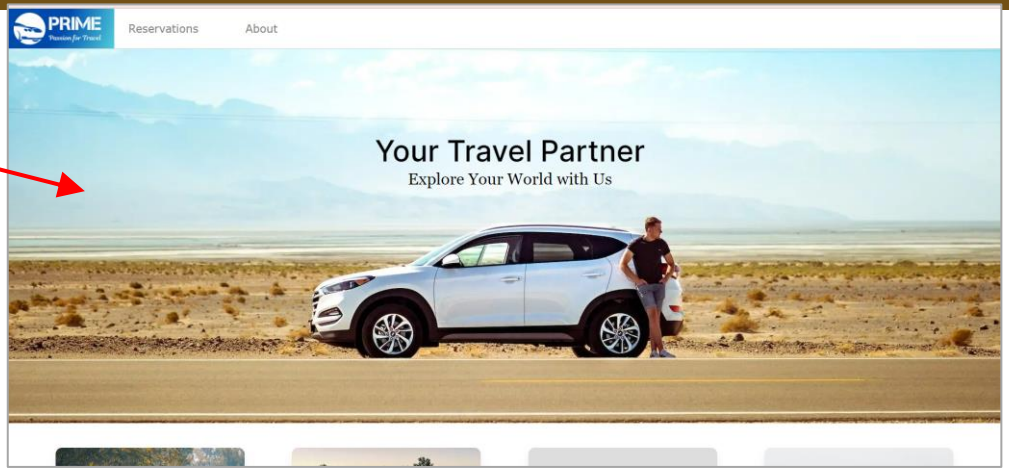
- State Management เป็นกระบวนการที่สำคัญ เนื่องจากเกี่ยวข้องกับ data ที่ประมวลผลและอาจกระทบการแสดงผลข้อมูล ใน session การใช้งานของ user
  - Next.js สนับสนุนการเข้าถึงและจัดการ state ได้หลากหลายวิธี



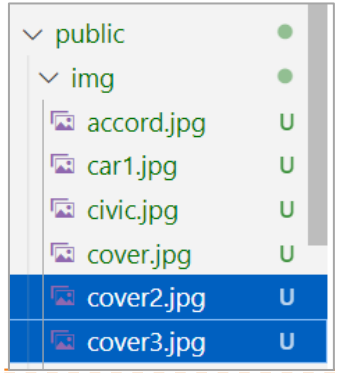
Workshop 6.1 :
Keeping Data in Local Variable

Keeping Data in Local Variable

- เปลี่ยนรูป Banner ตาม data ที่จัดเก็บอยู่ใน local variable ของ component?



- เพิ่มรูป `cover2.jpg` และ `cover3.jpg`





Keeping Data in Local Variable



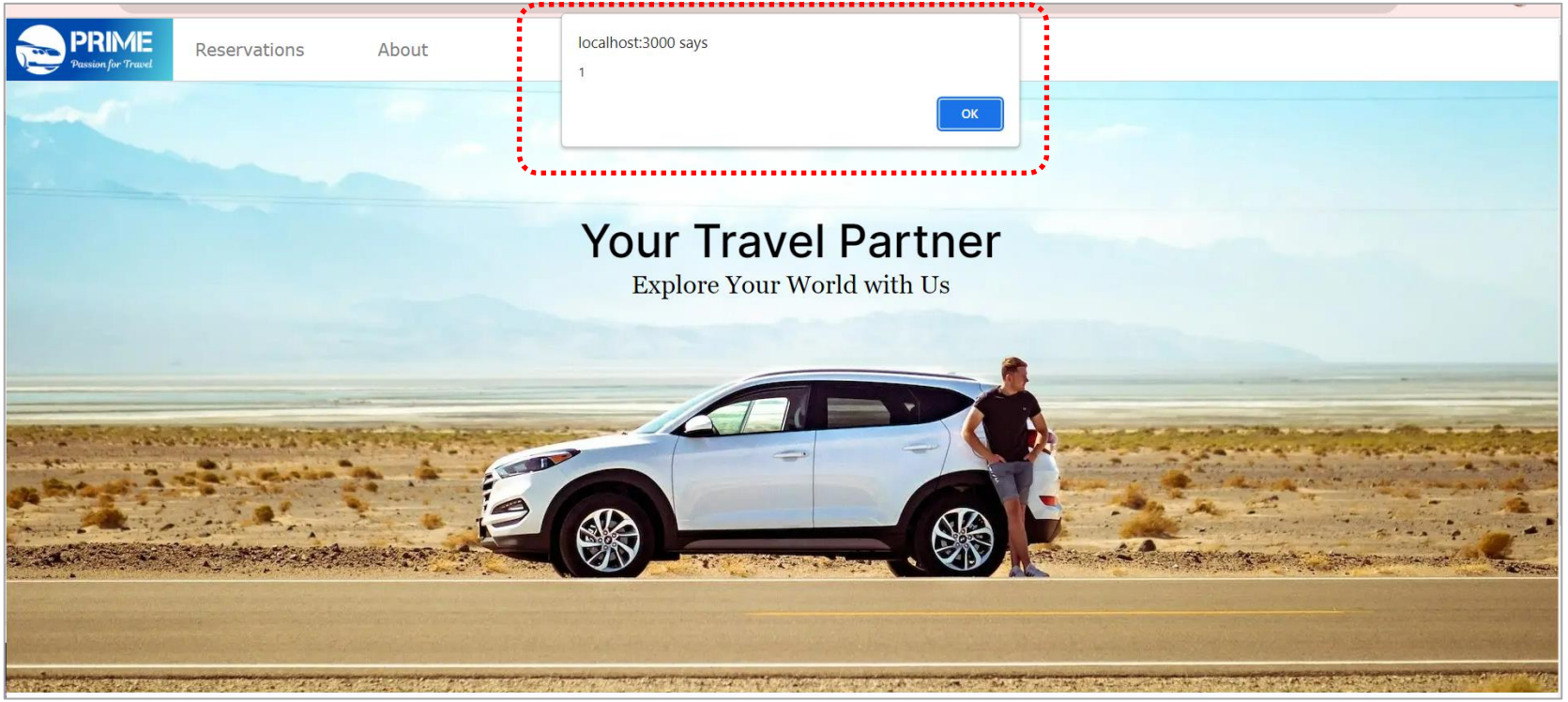
- components/Banner.tsx ให้เก็บ path ไปยังรูป cover ไว้ใน array และ update index ของ array เมื่อคลิก Banner

```
'use client'
import styles from './banner.module.css'
import Image from 'next/image';

export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    let index = 0
    return (
        <div className={styles.banner} onClick={()=>{index++; alert(index%3)}}>
            <Image src={covers[index%3]}
            alt='cover'
            fill={true}
            objectFit='cover'
            />
            <div className={styles.bannerText}>
                <h1 className='text-4xl font-medium'>Your Travel Partner</h1>
                <h3 className='text-xl font-serif'>Explore Your World with Us</h3>
            </div>
        </div>
    );
}
```

Keeping Data in Local Variable

- เมื่อคลิก local variable `index` เปลี่ยนค่า แต่ Image ไม่เปลี่ยน



State



- Local Variable ใน component มีข้อจำกัด
  - การเปลี่ยนค่า local variable ไม่ทำให้เกิดการ render component ใหม่ เนื่องจาก React ไม่ทราบว่าต้อง re-render เมื่อมีการเปลี่ยนแปลง data
  - แต่! หากมีการ re-render ใหม่ ค่า local variable เดิมจะไม่คงอยู่ เพราะ local variable ไม่รักษาค่าเดิมระหว่างการ re-render (non-persistent)

- ค่าอัปเดต component ด้วย data ใหม่ ต้องการ feature 2 อย่าง
  - การรักษาค่าของ data ข้ามการ re-render
  - การ trigger ให้ React ทำการ render component ใหม่ (re-render) เมื่อมีค่า data ใหม่

- React ใช้ `state` variable ในการทำงานนี้ โดยใช้งานได้ผ่าน `useState` Hook
  - `state` variable รักษาค่าของ data ข้ามการ re-render
  - state `setter` function อัปเดตค่าของ `state` variable และ trigger ให้ React re-render component

Hooks



- React มาพร้อมชุดของ function ที่เรียกว่า Hooks ที่ใช้ในการเพิ่ม logic การทำงานเพิ่มเติมเข้าไปให้กับ component
  - เป็น function ที่พร้อมใช้งานเฉพาะเมื่อ React กำลัง render และใช้เพื่อเกี่ยว (hook) เข้ากับ React Features ต่าง ๆ
- Function เหล่านี้ขึ้นต้นด้วยคำว่า `use`
  - React มาพร้อม built-in Hooks เช่น `useState` , `userReducer`, `useEffect` เป็นต้น
  - Developer สามารถ customize Hooks ได้ โดย combine hook ที่มีอยู่
- Hooks เป็นฟังก์ชั่นที่มีเงื่อนไขเข้มงวดกว่า functions ทั่วไป
  - เรียก Hooks ได้จาก top level ของ component (หรือ Hooks อื่น) เท่านั้น
    - ไม่ควรเรียก hook ได้ภายในเงื่อนไข หรือใน loop หรือ nested function เนื่องจากลำดับของการ execute hook อาจผิดไป เมื่อ component ถูก render
    - แม้ว่า hook เป็น function แต่ทำงานคล้าย unconditional declaration ที่ประกาศถึง feature ที่ component นั้น ๆ ต้องการ

useState Hook



- `useState` Hook สนับสนุนให้ component จดจำ data
  - จำ input value จาก form
  - จำ index ของภาพที่เลือกใน gallery
- ใช้ `useState` ในการประกาศ state variable ที่สามารถอัปเดตค่าได้โดยตรง
  - รับค่าตั้งต้นของ state

```
const [stateVal, setterFunction] = useState(initialState)
```

    - *initialState* ค่าเริ่มต้นของ state variable
  - คืนค่าเป็น array ที่มีสมาชิก 2 ตัว นิยมใช้ array destructuring [ ] มารับค่าเพื่ออ่านค่าจาก array มาใช้งานได้ทันที คือค่า state ปัจจุบัน และ state setter function
    - *stateVal* state variable เก็บค่า state
    - *setterFunction* ชื่อ setter function
  - ตัวอย่างการประกาศ state `status` มีค่าเริ่มต้นเป็น *typing* และมี setter เป็นฟังก์ชั่น `setStatus`

```
const [status, setStatus] = useState('typing')
```

useState Hook



- set function จะอัปเดต state variable เป็นค่าใหม่สำหรับการ render ครั้งถัดไป
  - หากอ่านค่า state variable หลังการเรียก set function จะได้ค่าเดิม

- หากค่าใหม่ของ state variable เท่ากับค่าปัจจุบัน (เมื่อเปรียบเทียบด้วย `Object.is`), React จะข้ามการ re-render component รวมถึง child ของ component ด้วย โดยเป็นส่วนหนึ่งของการทำ optimization

- หากมีการทำ state update เป็น batch โดย update state variable หลายตัวต่อ 1 event เพื่อป้องกันการ re-render หลายครั้งต่อ event โดย React จะอัปเดต screen หลังการ run ทุก event handler แล้วและเรียก setter function ต่าง ๆ แล้ว

- การเรียก set function ระหว่างการ render เป็นไปได้เฉพาะเมื่อเรียกจาก component ที่กำลังถูก render อยู่เท่านั้น โดย React จะทิ้ง output แล้ว render ใหม่ด้วย state ใหม่

State



- **Local Variable ใน component มีข้อจำกัด**
  - การเปลี่ยนค่า local variable ไม่ทำให้เกิดการ render component ใหม่ เนื่องจาก React ไม่ทราบว่าต้อง re-render เมื่อมีการเปลี่ยนแปลง data
  - แต่! หากมีการ re-render ใหม่ ค่า local variable เดิมจะไม่คงอยู่ เพราะ local variable ไม่รักษาค่าเดิมระหว่างการ re-render (non-persistent)

- **ค่าอัปเดต component ด้วย data ใหม่ ต้องการ feature 2 อย่าง**
  - การรักษาค่าของ data ข้ามการ re-render
  - การ trigger ให้ React ทำการ render component ใหม่ (re-render) เมื่อมีค่า data ใหม่

- **React ใช้ `state` variable ในการทำงานนี้ โดยใช้งานได้ผ่าน `useState` Hook**
  - `state` variable รักษาค่าของ data ข้ามการ re-render
  - state `setter` function อัปเดตค่าของ `state` variable และ trigger ให้ React re-render component

Hooks



- React มาพร้อมชุดของ function ที่เรียกว่า Hooks ที่ใช้ในการเพิ่ม logic การทำงานเพิ่มเติมเข้าไปให้กับ component
  - เป็น function ที่พร้อมใช้งานเฉพาะเมื่อ React กำลัง render และใช้เพื่อเกี่ยว (hook) เข้ากับ React Features ต่าง ๆ
- Function เหล่านี้ขึ้นต้นด้วยคำว่า `use`
  - React มาพร้อม built-in Hooks เช่น `useState` , `userReducer`, `useEffect` เป็นต้น
  - Developer สามารถ customize Hooks ได้ โดย combine hook ที่มีอยู่
- Hooks เป็นฟังก์ชั่นที่มีเงื่อนไขเข้มงวดกว่า functions ทั่วไป
  - เรียก Hooks ได้จาก top level ของ component (หรือ Hooks อื่น) เท่านั้น
    - ไม่ควรเรียก hook ได้ภายในเงื่อนไข หรือใน loop หรือ nested function เนื่องจากลำดับของการ execute hook อาจผิดไป เมื่อ component ถูก render
    - แม้ว่า hook เป็น function แต่ทำงานคล้าย unconditional declaration ที่ประกาศถึง feature ที่ component นั้น ๆ ต้องการ

useState Hook



- `useState` Hook สนับสนุนให้ component จดจำ data
  - จำ input value จาก form
  - จำ index ของภาพที่เลือกใน gallery
- ใช้ `useState` ในการประกาศ state variable ที่สามารถอัปเดตค่าได้โดยตรง
  - รับค่าตั้งต้นของ state

```
const [stateVal, setterFunction] = useState(initialState)
```

    - *initialState* ค่าเริ่มต้นของ state variable
  - คืนค่าเป็น array ที่มีสมาชิก 2 ตัว นิยมใช้ array destructuring [ ] มารับค่าเพื่ออ่านค่าจาก array มาใช้งานได้ทันที คือค่า state ปัจจุบัน และ state setter function
    - *stateVal* state variable เก็บค่า state
    - *setterFunction* ชื่อ setter function
  - ตัวอย่างการประกาศ state `status` มีค่าเริ่มต้นเป็น *typing* และมี setter เป็นฟังก์ชั่น `setStatus`

```
const [status, setStatus] = useState('typing')
```

useState Hook

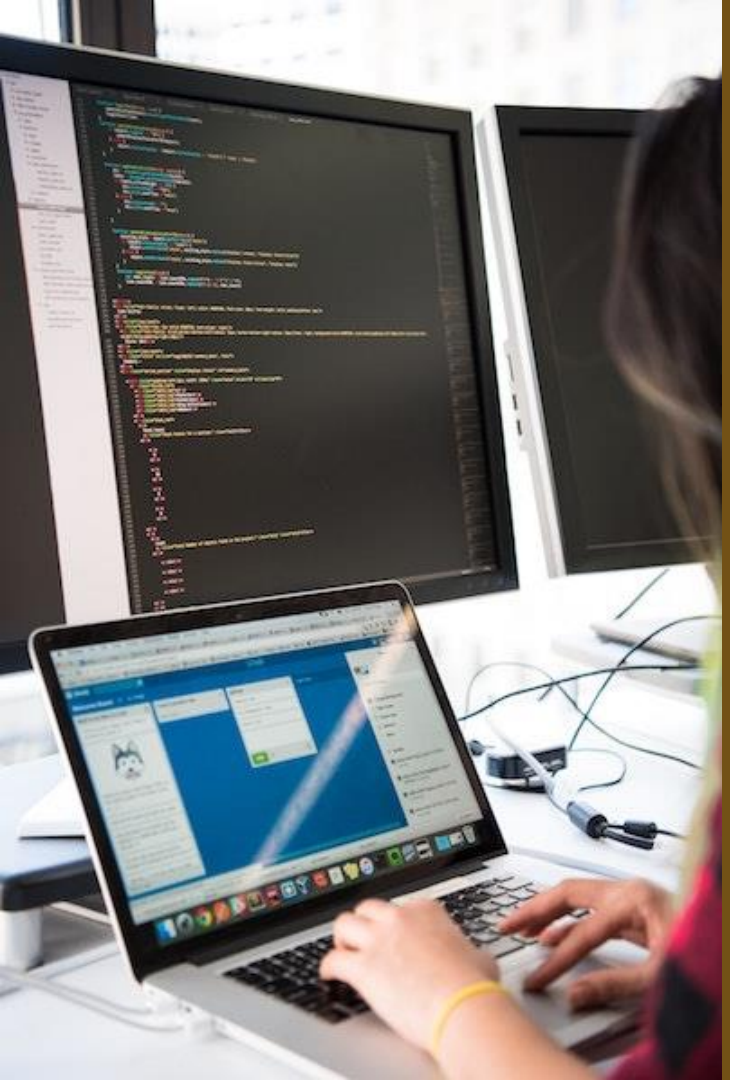


- set function จะอัปเดต state variable เป็นค่าใหม่สำหรับการ render ครั้งถัดไป
  - หากอ่านค่า state variable หลังการเรียก set function จะได้ค่าเดิม

- หากค่าใหม่ของ state variable เท่ากับค่าปัจจุบัน (เมื่อเปรียบเทียบด้วย `Object.is`), React จะข้ามการ re-render component รวมถึง child ของ component ด้วย โดยเป็นส่วนหนึ่งของการทำ optimization

- หากมีการทำ state update เป็น batch โดย update state variable หลายตัวต่อ 1 event เพื่อป้องกันการ re-render หลายครั้งต่อ event โดย React จะอัปเดต screen หลังการ run ทุก event handler แล้วและเรียก setter function ต่าง ๆ แล้ว

- การเรียก set function ระหว่างการ render เป็นไปได้เฉพาะเมื่อเรียกจาก component ที่กำลังถูก render อยู่เท่านั้น โดย React จะทิ้ง output แล้ว render ใหม่ด้วย state ใหม่



Workshop 6.2 :
Using State variable

Using State variable



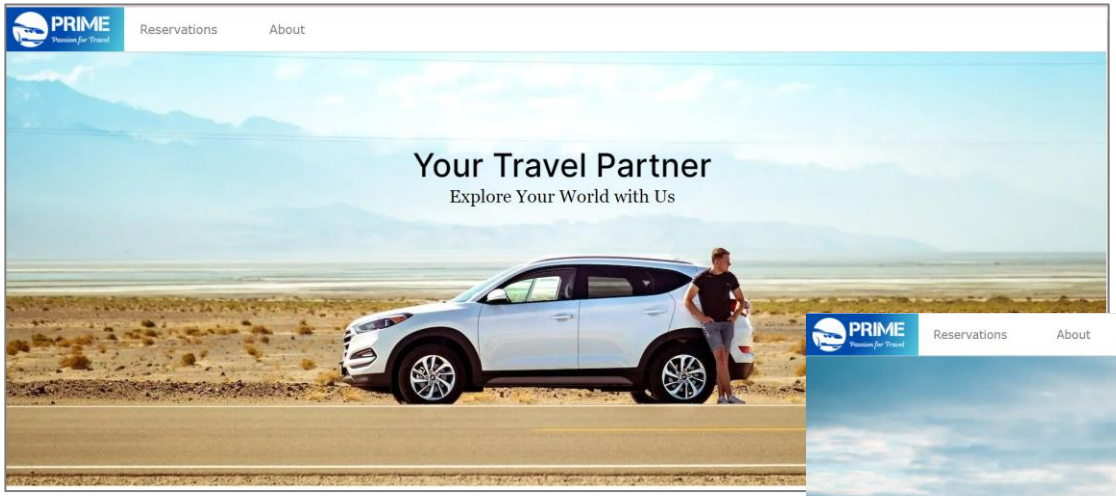
- ใน Banner.tsx
  เก็บ index ของ array ไว้ใน state variable และเรียก setter function เมื่อคลิก Banner

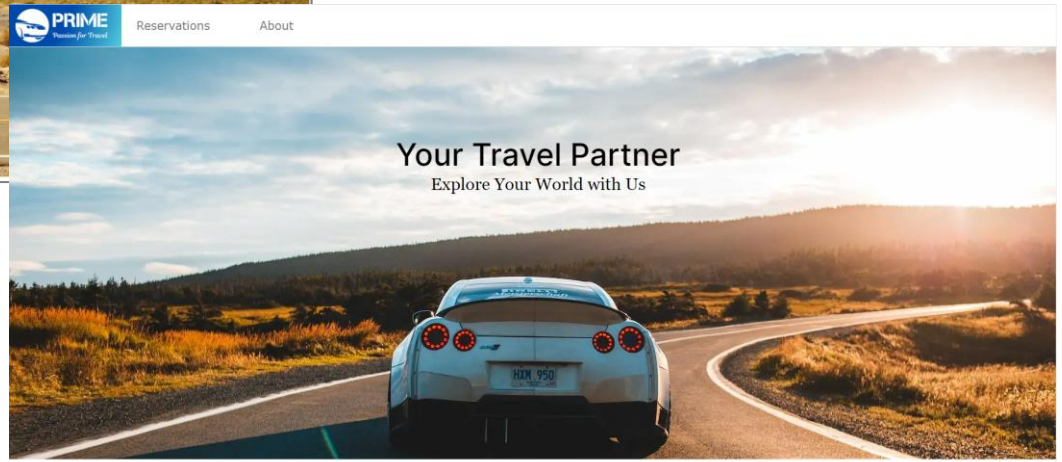
```tsx
'use client'
import { useState } from 'react';
import styles from './banner.module.css'
import Image from 'next/image';

export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    return (
        <div className={styles.banner} onClick={()=>setIndex(index+1)}>
            <Image src={covers[index%3]}
            alt='cover'
            fill={true}
            objectFit='cover'
            />
            <div className={styles.bannerText}>
                <h1 className='text-4xl font-medium'>Your Travel Partner</h1>
                <h3 className='text-xl font-serif'>Explore Your World with Us</h3>
            </div>
        </div>
    );
}
```

Using State variable

- เมื่อคลิก state variable `index` เปลี่ยน และ component ถูก re-render พร้อม Image ที่อัปเดต





useReducer Hook



- useReducer hook ใช้ในการบริหารจัดการ state ผ่าน reducer function
- reducer function เป็น function ที่รวมเอา logic สำหรับการ update state ออกมาไว้ใน function
  - เหมาะกับ component ที่มีรูปแบบ logic สำหรับ update state variable หลายแบบ ซึ่ง logic นั้นอาจกระจัดกระจายอยู่ทั่ว component
  - Reducer function รับ 2 argument โดยมีรูปแบบต่อไปนี้

```
function reducerFuncName(stateVal, action){
   … return updated value of stateVal …
}
```

```
(stateVal, action)=>{
   …
}
```

    - *stateVal* ค่าปัจจุบันของ state variable
    - *action* action object ที่บรรจุ data ที่ใช้ประกอบ logic ในการ update ค่า state variable
    - Return ค่าที่อัปเดตแล้วของ state variable
  - Reducer function ทำงานกับ dispatch function ซึ่งแจก (dispatch) ข้อมูล *action* ไปยัง Reducer function โดยปกติจะถูกเรียกโดย event handler

```
dispatchFuncName(action)
```

useReducer Hook

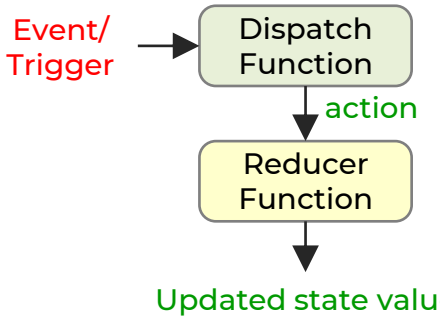


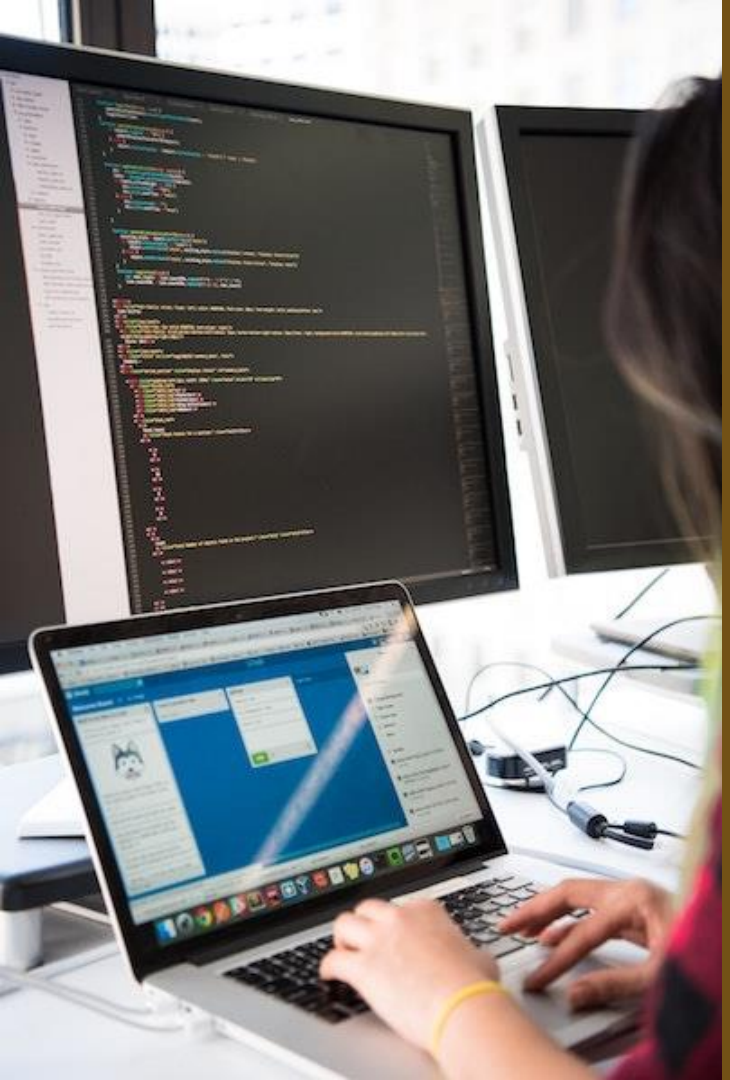
- ใช้ useReducer ในการประกาศ ด้วยรูปแบบต่อไปนี้

```
const [stateVal, dispatchFuncName] = useReducer(reducerFuncName, initialArg, initFunc?)
```

- *reducerFuncName* Reducer function
- *initialArg* ค่า argument สำหรับ *initFunc* ที่ใช้คำนวณค่าเริ่มต้นของ state variable
  หากไม่มีการระบุ *initFunc* ค่านี้จะถูกใช้เป็นค่าเริ่มต้นของ state variable
- *initFunc* Function สำหรับใช้ในการคำนวณค่าเริ่มต้นของ state variable

- คืนค่าเป็น array ที่มีสมาชิก 2 ตัว นิยมใช้ array destructuring [ ] มารับค่าเพื่ออ่านค่าจาก array มาใช้งานได้ทันที คือค่า state ปัจจุบัน และ state setter function
  - *stateVal* state variable เก็บค่า state
  - *dispatchFuncName* ชื่อ dispatch function





Workshop 6.3 :
Using Reducer Function

Using Reducer Function



แยกส่วนแสดงรายการ Car ซึ่งใช้งาน Reducer และจะเป็น client component ออกมาจาก page โดย

- สร้าง component `component/CarPanel.tsx` บรรจุรายการของ `ProductCard` Component

```
import ProductCard from "./ProductCard"
export default function CarPanel() {

    return (
        <div style={{width: "100%"}}>
            <div style={{margin:"20px", display:"flex", width: "100%",
            flexDirection:"row", alignContent:"space-around",
            justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
                <ProductCard carName='Honda Civic' imgSrc='/img/civic.jpg'/>
                <ProductCard carName='Honda Accord' imgSrc='/img/accord.jpg'/>
                <ProductCard carName='Toyota Fortuner' imgSrc='/img/fortuner.jpg'/>
                <ProductCard carName='Tesla Model 3' imgSrc='/img/tesla.jpg'/>
            </div>
        </div>
    )
}
```

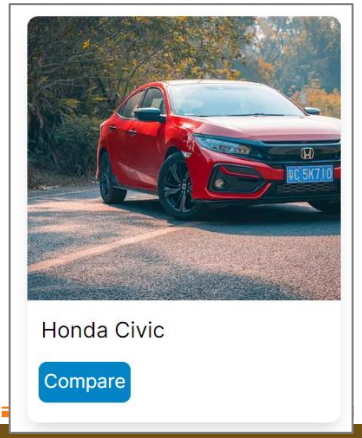
Using Reducer Function

- ที่หน้า component `app/Page.tsx` ใช้งาน `CarPanel` Component

```tsx
import Banner from "@/components/Banner";
import CarPanel from '@/components/CarPanel'

export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
        <Banner/>
        <CarPanel/>
      </main>
    </div>
  );
}
```

- ที่ `component/ProductCard.tsx` เพิ่มปุ่มสำหรับเพิ่ม car นั้น ๆ เข้าไปในการเปรียบเทียบข้อมูล




```tsx
<InteractiveCard contentName={carName}>
    <div className='w-full h-[70%] relative rounded-t-lg'>
    <Image src={imgSrc}
        alt='Product Picture'
        fill={true}
        className='object-cover rounded-t-lg'/>
    </div>
    <div className='w-full h-[15%] p-[10px]'>{carName}</div>
    <button
    className="block text-sm h-[10%] rounded-md bg-sky-600
    hover:bg-indigo-600 mx-2 px-1 py-1
    shadow-sm text-white">Compare</button>
</InteractiveCard>
```

Using Reducer Function



- ที่ component/carPanel.tsx เพิ่ม reducer function สำหรับบริหารจัดการ state variable compareList ซึ่งเป็น set ของชื่อรถที่จะถูกนำมาเปรียบเทียบกัน

```tsx
'use client'
import { useReducer } from 'react'
import ProductCard from '@/components/ProductCard'

export default function CarPanel() {
    const compareReducer = (compareList:Set<string>, action:{type:string, carName:string})=>{
        switch(action.type) {
          case 'add': {
            return new Set(compareList.add(action.carName)) }
          case 'remove': {
            compareList.delete(action.carName)
            return new Set(compareList)
          }
          default: return compareList
        }
    }

    const [compareList, dispatchCompare] = useReducer(compareReducer, new Set<string>())

    return(
```

ถ้าเป็นการ add จะเป็นการเพิ่มชื่อรถเข้ามาใน set
ถ้าเป็นการ remove จะลบชื่อรถออกจาก set

Using Reducer Function



- ที่ component/ProductCard.tsx รับ Function onCompare เข้ามา สำหรับเรียก เมื่อมีการกดปุ่ม Compare เพิ่ม car นั้น ๆ เข้าไปในการเปรียบเทียบข้อมูล

```
export default function ProductCard ( { carName, imgSrc, onCompare }
    : { carName:string, imgSrc:string, onCompare:Function}) {
    return (
        <InteractiveCard contentName={carName}>
            <div className='w-full h-[70%] relative rounded-t-lg'>
            <Image src={imgSrc}
                alt='Product Picture'
                fill={true}
                className='object-cover rounded-t-lg'/>
            </div>
            <div className='w-full h-[15%] p-[10px]'>{carName}</div>
            <button
            className="block text-sm h-[10%] rounded-md bg-sky-600
            hover:bg-indigo-600 mx-2 px-1 py-1
            shadow-sm text-white"
            onClick={(e)=>{e.stopPropagation(); onCompare(carName)}}
            >Compare</button>
        </InteractiveCard>
    );
```

Using Reducer Function



- ที่ component/CarPanel.tsx ส่งการทำงานสำหรับ onCompare ให้เรียก dispatch function ของ reducer โดยส่ง
  - type ที่ประกอบไปด้วยชนิดการทำงานกับ set เป็น add
  - carName ชื่อชื่อรถที่จะ add

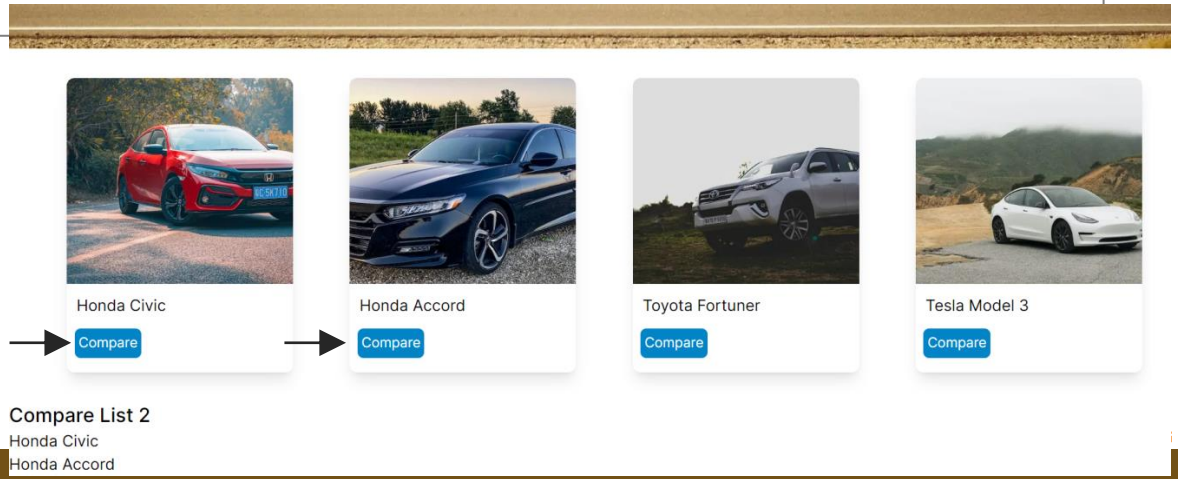
```
flexDirection:"row", alignContent:"space-around",
justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
    <ProductCard carName='Honda Civic' imgSrc='/img/civic.jpg'
    onCompare={(car:string)=>dispatchCompare({type:'add', carName:car})}
    />
    <ProductCard carName='Honda Accord' imgSrc='/img/accord.jpg'
    onCompare={(car:string)=>dispatchCompare({type:'add', carName:car})}
    />
    <ProductCard carName='Toyota Fortuner' imgSrc='/img/fortuner.jpg'
    onCompare={(car:string)=>dispatchCompare({type:'add', carName:car})}
    />
    <ProductCard carName='Tesla Model 3' imgSrc='/img/tesla.jpg'
    onCompare={(car:string)=>dispatchCompare({type:'add', carName:car})}
    />
</div>
```

Using Reducer Function

- ที่ component/CarPanel.tsx เพิ่มส่วนการแสดงข้อมูลใน compareList ไว้ด้านล่าง
  - ใช้ function map ของ Array เพิ่ม map สมาชิกของ array เข้ามาสร้างเป็น JSX component

```
            <ProductCard carName='Tesla Model 3' imgSrc='/img/tesla.jpg'
            onCompare={(car:string)=>dispatchCompare({type:'add', carName:car})}
            />
        </div>
        <div className="w-full text-xl font-medium">Compare List {compareList.size}</div>
            { Array.from(compareList).map((car)=><div key={car}>{car}</div>) }
    </div>
```

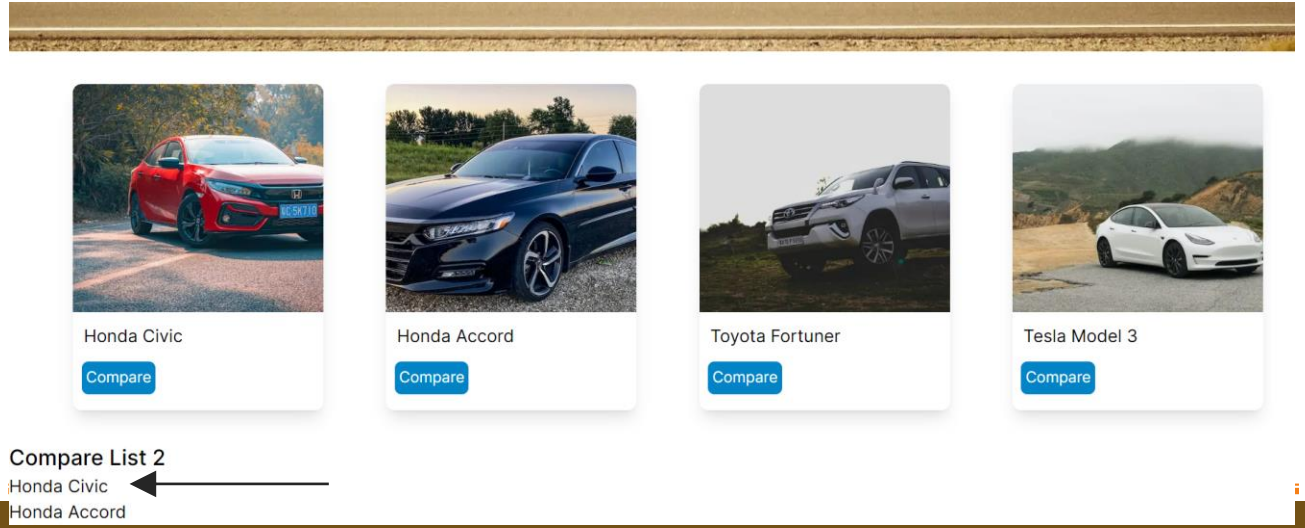
- ลองเพิ่ม car เข้าไปใน compare list

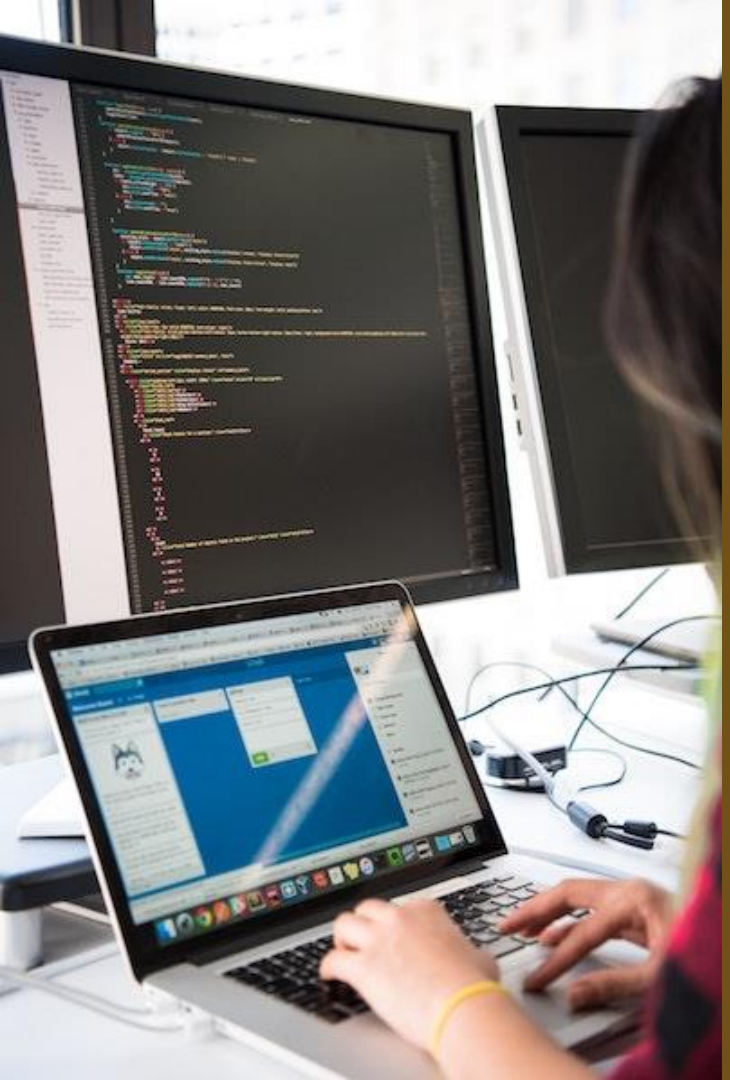


Using Reducer Function

- ให้เมื่อคลิกที่ชื่อรถใน compare list เป็นการนำชื่อรถนั้น ๆ ออกจาก list ผ่านการเรียก dispatcher function

```
    </div>
    <div className="w-full text-xl font-medium">Compare List {compareList.size}</div>
        { Array.from(compareList).map((car)=><div key={car}
        onClick={()=>dispatchCompare({type:'remove', carName:car})}
        >{car}</div>) }
</div>
```





Workshop 6.4 : Controlled Component with State

Make Controlled Component with State



- ที่ component /LocationDate Reserve.tsx เพิ่มการใช้งาน state เพื่อให้ form component เป็น controller component

```tsx
import { Select, MenuItem } from "@mui/material"
import { useState } from "react"
import { Dayjs } from 'dayjs'

export default function LocationDateReserve() {

    const [reserveDate, setReserveDate] = useState<Dayjs|null>(null)
    const [location, setLocation] = useState('BKK')

    return (
        <div className="bg-slate-100 rounded-lg gap-x-5 gap-y-2
        w-fit px-10 py-5 flex flex-row justify-center">
            <LocalizationProvider dateAdapter={AdapterDayjs}>
                <DatePicker className="bg-white"
                value={reserveDate} onChange={(value)=>setReserveDate(value)}/>
            </LocalizationProvider>
            <Select variant="standard"
            name="location" id="location" value={location}
            onChange={ (e)=>setLocation(e.target.value) }
            className="h-[2em] w-[200px]">
                <MenuItem value="BKK">Bangkok</MenuItem>
                <MenuItem value="CNX">Chiang Mai</MenuItem>
                <MenuItem value="HKT">Phuket</MenuItem>
            </Select>
        </div>
    )
}
```